Uppsala University



Machine Learning

Assignment 2

Contents

1 Oral exam question contribution 5

2 Decision Trees 6

3 Running standard tools for boosting and random forests 14

4 *Bagged tree and random forest implementation 15

General information

- The recommended tool in this course is R (with the IDE R-Studio). You can download R **here** and R-Studio **here**. You can use Python and Jupyter Notebooks, although the assignments may use data available only through the R package, a problem you would need to solve yourself.
- Report all results in a single, `*.pdf`-file. *Other formats, such as Word, Rmd, Jupyter Notebook, or similar, will automatically be failed.* Although, you are allowed first to knit your document to Word or HTML and then print the assignment as a PDF from Word or HTML if you find it difficult to get TeX to work.
- The report should be written in English.
- If a question is unclear in the assignment. Write down how you interpret the question and formulate your answer.
- You should submit the report to **Studium**. Deadlines for all assignments are **Sunday 23.59**. See **Studium** for dates. Assignments will be graded within 10 working days from the assignment deadline.
- To pass the assignments, *you should answer all questions not marked with **, and get at least 75% correct.
- To get VG on the assignment, *also the questions marked with a ** should be answered. Sometimes you can choose between different VG assignments, then this is explicitly stated. To get VG you need 75% both on questions to pass and the VG part of the assignment. VG will only be awarded on the first deadline of the assignment.
- A report that does not contain the general information (see the **template**), will be automatically rejected.
- Every assignment report must include one oral exam question of your own together with a short model answer, following the oral exam question contribution instructions in this assignment. You are not allowed to use large language models, such as ChatGPT, at all for this contribution. State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.
- When working with R, we recommend writing the reports using R markdown and the provided **R markdown template**. The template includes the formatting instructions and how to include code and figures.
- Instead of R markdown, you can use other software to make the pdf report, but you should use the same instructions for formatting. These instructions are also available as **a PDF produced from the R markdown template**.
- The course has its own R package `uuml` with data and functionality to simplify coding. To install the package just run the following:

  1. `install.packages("remotes")`
  2. ```
     remotes::install_github("MansMeg/IntroML",
     subdir = "rpackage")
     ```
     ```

- Many of the exercises can be checked automatically using the R package `markmyassignment`. Information on how to install and use the package can be found **here**. *Don't include* ***`markmyassignment`*** *results in the report.*
- We collect common questions regarding installation and technical problems in a course Frequently Asked Questions (FAQ). This can be found **here**.
- You may use large language models (LLMs), such as ChatGPT, like you may discuss with fellow students: to learn, ask for explanations, and get feedback. You may not use LLMs to produce the solution you submit. You must be able to explain every part of your own assignment at the oral exam, including your code, results, and reasoning, even when it is not covered by the published oral exam questions. If course staff have concerns about your understanding or the authorship of any part of your submission, you may be asked to explain that part at the oral exam.
- For optional LLM study support, first upload or paste the `AI_TUTOR.md` instructions into a new conversation and ask the tool to follow them. You may then upload the full teacher-provided assignment instructions, including supplied example code. Ask for explanations, incremental hints, and separate practice examples, not completed solutions. You remain responsible for following the course rules even if the tool offers a solution.
- Do not upload your own report, answers, or assessed code to an LLM. Do not share your solutions (text or code) with other students, but feel free to discuss concepts and approaches. Sharing your solutions may also be considered cheating. During scheduled support sessions held on Zoom, only share your screen in a separate room with a teaching assistant.
- Where an exercise explicitly asks you to run an LLM and report its output, you may do so for that experiment. Your explanations and analysis must be your own. This permission does not apply to the oral exam question contribution, for which LLM use is prohibited entirely.
- Scheduled support sessions are for questions about the assignments and course material. Do not hesitate to ask! Ask questions early if instructions are unclear or you encounter possible bugs, so you do not lose time unnecessarily.
- If you have any suggestions or improvements to the course material, please post in the course chat feedback channel, create an issue **here**, or submit a pull request to the public repository.

Automatic feedback with `markmyassignment`

The R package `markmyassignment` (MMA) provides automatic feedback on the functions listed in the selected check configuration. Define or load your own functions in R before running the checks. Use the `tasks` argument to check one function by name; omitting it checks every function in the selected configuration. Functions that depend on other functions need those definitions loaded too.

To select the checks for this assignment, run the following code in R:

```
install.packages("markmyassignment") # Install once
library(markmyassignment)
assignment_path <-
  paste("https://github.com/MansMeg/IntroML/",
        "blob/master/assignments/tests/assignment2/assignment2.yml", sep="")
set_assignment(assignment_path)
# List the function names available in this configuration
show_tasks()
# To check one function, replace function_name with its name:
# mark_my_assignment(tasks = "function_name")
# To check your code/functions, just run
mark_my_assignment()
```

The checks provide feedback on selected tasks. They do not assess the whole assignment or award marks. Print your complete function definitions in the PDF report as required by the general instructions, even when the checks pass. *You should not need to include MMA results in the report.* The supplied checks run R code. Python implementations are assessed from the report and cannot be run directly through MMA.

---

1 Oral exam question contribution

Write one possible oral exam question of your own for this assignment together with a short model answer. Include both in your assignment report.

The question should:

- concern material from this assignment or the associated reading or lectures,
- be suitable for a 1–3 minute oral answer, possibly using a whiteboard,
- test conceptual understanding rather than only calculation or code,
- not already be included in the oral exam question list and not be very close to an existing question (e.g. just different numbers).

Note! You are not allowed to use large language models, such as ChatGPT, at all for this oral exam question contribution.

State clearly whether the course staff may use or adapt your question and answer in a future oral exam: **Yes** or **No**.

2 Decision Trees

The dataset `Hitters` contain data on the salary of Baseball players and additional information on years in the baseball league (`Years`) and the number of hits (`Hits`). To access the data, use:

```
library(uuml)
data("Hitters")
```

We are now going to build an algorithm to grow decision trees. The algorithm used below is based on the algorithm explained in Chapter 9.2.2 in Hastie et al. (2009). To split a tree, see Eq. (9.12-9.14).

1. Create a test set by setting aside the 30 first observations. Remove those observations from your training set.

   ```
   # Remove NA values
   Hitters <- Hitters[complete.cases(Hitters),]

   # Create test and training set
   X_test <- as.matrix(Hitters[1:30, c("Years", "Hits"), drop = FALSE])
   y_test <- Hitters[1:30, c("Salary")]
   X_train <- as.matrix(Hitters[31:nrow(Hitters), c("Years", "Hits"),
                                drop = FALSE])
   y_train <- Hitters[31:nrow(Hitters), c("Salary")]
   ```

   The predictor objects `X_train`, `X_test`, and the example predictors below are numeric matrices; the responses remain numeric vectors. When selecting rows or columns of a predictor matrix, use `drop = FALSE` to retain both dimensions, including for one observation or one predictor. For example, use `X[rows, cols, drop = FALSE]`.

2. (3p) **Implementation (2p).** Implement an R function to split observations binary greedily. Use the complete-case data prepared above; additional missing-value handling is not required. Here we use the Hastie et al. (2009, Eq. 9.12-9.14), but with the addition that each child node must contain at least l observations. The function should take a design matrix `X`, an `y` variable and a minimal leaf size `l`. The function should return a list with the index set of the observations in `R1`, `R2`, the split value `s` and the covariate used for the split `j`, or `NULL` if no valid split exists. Below is a rough sketch how you can implement the function.

   Demonstration (1p). Reuse one of the example calls below or the first split in Task 1.3, and briefly explain the actual output from your function.

   ```
   tree_split <- function(X, y, l){
     checkmate::assert_matrix(X)
     checkmate::assert_numeric(y, len = nrow(X))
     checkmate::assert_int(l)

     # Store the sum of squares
   ```

```
  SS <- matrix(Inf, nrow = nrow(X), ncol = ncol(X))
  for(j in 1:ncol(X)){
    for(k in 1:nrow(X)){
      s <- X[k,j] # Choose split point
      R1 <- which(X[,j] <= s)
      R2 <- which(X[,j] > s)

      # Do if R1 or R2 is smaller than the leaf size l
      # Compute c1
      # Compute c2
      SS[k, j] <- stop("TODO: compute the sum of squares")
    }
  }

  # No candidate split satisfies the minimum child size.
  if(!any(is.finite(SS))) return(NULL)

  # The final results for the min SS (arg min)
  j <- stop("TODO: choose the split variable")
  s <- stop("TODO: choose the split value")
  R1 <- which(X[,j] <= s)
  R2 <- which(X[,j] > s)
  list(j = j,
       s = s,
       R1 = unname(R1),
       R2 = unname(R2),
       SS = min(SS))
}
```

Hint 1! Several candidate splits may have the same minimum SS. Any valid minimising split is acceptable; the example code selects the first minimum:

```
which(SS == min(SS), arr.ind = TRUE)[1, ]
```

Repeated predictor values are a separate issue: observations with the same value must stay together when splitting that predictor.

Hint 2! Leave `SS` equal to `Inf` for candidates with fewer than l observations in either child. If no finite score remains, `tree_split()` must return `NULL`. Having at least $2l$ observations does not guarantee a valid split: predictors may be constant, or repeated predictor values may prevent a sufficiently balanced split. Whenever a split is `NULL`, `grow_tree()` must create a leaf whose `gamma` is the mean response in that node, including when the root cannot split. Remove each processed node from the queue, including leaf nodes, so the loop makes progress.

See below for an example how the function should work.

```
# Lets choose a small data to try out our algorithm with
X_check <- as.matrix(Hitters[31:50, c("Years", "Hits"), drop = FALSE])
y_check <- Hitters[31:50, c("Salary")]
# These are the names of the players we look at
rownames(Hitters)[31:50]
```

```
##  [1] "-Bob Melvin"       "-BillyJo Robidoux" "-Bill Schroeder"
##  [4] "-Chris Bando"      "-Chris Brown"      "-Carmen Castillo"
##  [7] "-Chili Davis"      "-Carlton Fisk"     "-Curt Ford"
## [10] "-Carney Lansford"  "-Chet Lemon"       "-Candy Maldonado"
## [13] "-Carmelo Martinez" "-Craig Reynolds"   "-Cal Ripken"
## [16] "-Cory Snyder"      "-Chris Speier"     "-Curt Wilkerson"
## [19] "-Dave Anderson"    "-Don Baylor"
```

```
# This is how it should work
tree_split(X_check, y_check, l = 5)
```

```
## $j
## [1] 1
##
## $s
## [1] 5
##
## $R1
##  [1]  1  2  3  5  6  9 13 16 18 19
##
## $R2
##  [1]  4  7  8 10 11 12 14 15 17 20
##
## $SS
## [1] 1346633
```

```
# We can also make a split without any limit on leaf size
tree_split(X_check, y_check, l = 1)
```

```
## $j
## [1] 2
##
## $s
## [1] 132
##
## $R1
##  [1]  1  2  3  4  5  6  8  9 11 12 13 14 16 17 18 19
##
## $R2
## [1]  7 10 15 20
##
## $SS
## [1] 904383.4
```

Check: `mark_my_assignment(tasks = "tree_split")`

3. (2p) Using `l=5`, report the predictor (name or column index), split value, and sum of squares (SS) for the first split of `X_train` and `y_train`. Obtain all three from one call to `tree_split()`.

4. (3p) **Implementation (2p).** Use the function `tree_split()` to create a function `grow_tree()` that takes the arguments `X`, `y`, and `l` and then build a decision tree.

The returned tree can be a `data.frame` that looks as below. This is just one example implementation, you are free to implement this however you want.

Demonstration (1p). Include a reproducible call and compact actual tree output, with a brief explanation. You may use the example below or a tree from the required experiments.

Reading hint: Revisit Section 9.2.2 in Hastie et al. (2009), especially the paragraph following Equation (9.14), which explains how to extend the first split into a complete regression tree. Apply the minimum-child-size and stopping rules specified in this assignment.

Hint 1! Use a list in R to store the sets of observation indices you are splitting up.

Hint 2! The indices returned by `tree_split()` refer to rows of the subset supplied to that call. Convert them back to row positions in the original `X`.

Example: Indices and pointers (toy case, $l = 1$) *No submission required*

A pending node contains original data rows 4, 9, and 15. A split returns local indices `R1 = c(1, 3)` and `R2 = 2`. Map them back as follows:

```
node_rows <- c(4L, 9L, 15L)
local_R1 <- c(1L, 3L)
local_R2 <- 2L
node_rows[local_R1] # Original data rows: 4, 15
node_rows[local_R2] # Original data row: 9
```

Write this left child of tree row 1 as tree row 2 and update its parent's `R1_i`:

```
pointers <- data.frame(R1_i = c(-1L, -1L),
                       R2_i = c(-1L, -1L))
pointers$R1_i[1] <- 2L
pointers

##   R1_i R2_i
## 1    2   -1
## 2   -1   -1
```

Here 2 points to a *tree* row; 4, 9, and 15 identify *data* rows. A pointer of -1 marks an unwritten child. Each queue entry needs its data rows, parent tree row, and pointer to fill.

Suppose the root's right child (`other_rows`) is already waiting. This same processing step changes the queue as follows:

Stage	Queue position	Original data rows	Parent tree row	Pointer to fill
Before	1	$\{4, 9, 15\}$	1	`R1_i`
	2	`other_rows`	1	`R2_i`
After	1	`other_rows`	1	`R2_i`
	2	$\{4, 15\}$	2	`R1_i`
	3	$\{9\}$	2	`R2_i`

Remove the processed entry and append its children with parent tree row 2. The older entry stays first. Row 2's child pointers remain -1 until its children are written. A processed leaf is removed without adding children.

Here is an template how the function could be implemented, but you are free to implement it however you want.

```
grow_tree <- function(X, y, l=5){
  checkmate::assert_matrix(X)
  checkmate::assert_numeric(y, len = nrow(X))
  checkmate::assert_int(l)

  # We do an initial split
  init <- tree_split(X, y, l)
  if(is.null(init)){
    return(data.frame(j = NA, s = NA, R1_i = NA, R2_i = NA,
                      gamma = mean(y, na.rm = TRUE)))
  }
  # We use S_m to store the set of observation indicies
  # This is used to keep track of the groups of
  # observations to be split
  S_m <- list(init$R1, init$R2)
  # Storing the results
  # j = column to use for split
  # s = split point
  # R1_i = pointer to row where to go next if in R1
  # R2_i = pointer to row where to go next if in R2
  # gamma = the gamma value of the leaf (if j and s are NA)
  results <- data.frame(j = init$j,
                        s = init$s,
                        R1_i = -1,
                        R2_i = -1,
                        gamma = NA)
  while (length(S_m) > 0) {
    # As long as not all parts of the tree has been handled
    # we will either split or compute gamma
    node_split <- tree_split(X[S_m[[1]], , drop = FALSE],
                             y[S_m[[1]]], l)
    if(!is.null(node_split)){

      # Use node_split to fill j, s, R1 and R2.
      # Convert child indices to indices in the full data.

      # Add split point
      new_results <- data.frame(j = j,
                                s = s,
                                R1_i = -1,
                                R2_i = -1,
                                gamma = NA)
      results <- rbind(results,
                       new_results)
      # Add R1 and R2 to S_m
      S_m <- c(S_m, list(R1), list(R2))
      # Remove the set we just handled
      S_m[[1]] <- NULL
    } else {

      # Do stuff here to when you cant grow the tree (i.e. compute gamma)

      # Add leaf
      new_results <- data.frame(j = NA,
                                s = NA,
                                R1_i = NA,
```

```
                                R2_i = NA,
                                gamma = gamma)
      results <- rbind(results,
                       new_results)

      # Remove the set we just handled
      S_m[[1]] <- NULL
    }

  }
  return(results)
}
```

Here is an example how it should work.

```
tr <- grow_tree(X_check, y_check, l = 5)
tr
```

```
##    j   s R1_i R2_i gamma
## 1  1   5    2    3    NA
## 2  1   3    4    5    NA
## 3  2 101    6    7    NA
## 4 NA  NA   NA   NA 106.5
## 5 NA  NA   NA   NA 244.5
## 6 NA  NA   NA   NA 509.3
## 7 NA  NA   NA   NA 946.0
```

Check: `mark_my_assignment(tasks = "grow_tree")`

Run this check after also implementing `predict_with_tree()` in the next task. The check uses your growth and prediction functions together, so you can use your own tree representation.

5. (3p) **Implementation (2p).** Implement `predict_with_tree()`. Its arguments are `new_data` and `tree`. Here is a starting template for the function.

```
predict_with_tree <- function(new_data, tree){
  checkmate::assert_matrix(new_data)

  predictions <- numeric(nrow(new_data))

  for(i in seq_along(predictions)){
    not_in_leaf <- TRUE
    while(not_in_leaf){

      # Check the split and go to next row
      # until we end up with a gamma

    }
    predictions[i] <- predicted_value
  }
  return(predictions)
}
```

Demonstration (1p). Show a reproducible call and actual predictions, with a brief explanation. Reuse the example below or the test predictions from Task 1.6.

To use the tree to predict new observations X_{new} as follows.

```
X_new <- as.matrix(Hitters[51:52, c("Years", "Hits"), drop = FALSE])
X_new
```

```
##                 Years Hits
## -Daryl Boston       3   53
## -Darnell Coles      4  142
```

```
y_new <- Hitters[51:52, c("Salary")]
y_new
```

```
## [1]  75 105
```

```
pred_y <- predict_with_tree(new_data = X_new, tree = tr)
pred_y
```

```
## [1] 106.5 244.5
```

Check: `mark_my_assignment(tasks = "predict_with_tree")`

6. (1p) The function `rmse(x,y)` in the `uuml` R package computes the root mean squared error between `x` and `y`.

```
rmse
```

```
## function (x, y)
## {
##     checkmate::assert_numeric(x, any.missing = FALSE)
##     checkmate::assert_numeric(y, any.missing = FALSE, len = length(x))
##     sqrt(mean((x - y)^2))
## }
## <bytecode: 0x1378ea490>
## <environment: namespace:uuml>
```

```
rmse(x = c(75, 105, 33), y = c(102, 99, 43))
```

```
## [1] 16.98038
```

What is the root mean squared error (RMSE) on the test set for a tree trained on the whole training data `X_train` and `y_train`? Use `l=5`. Include this value in the comparison table requested in the next section.

3 Running standard tools for boosting and random forests

Use the predictor matrices `X_train`, `X_test` and response vectors `y_train`, `y_test` prepared in Section 2 for both models below. All three models then use the same training and test observations.

Collect the test RMSE for your single tree from the previous section, the `randomForest` model, and the XGBoost model in one comparison table with columns *Model* and *Test RMSE*. Add each result as you complete the tasks below. Present each RMSE once in this table and refer to it when comparing the models.

1. Use the `randomForest` R package to fit a random forest regression to the training data using the `randomForest` function.

```
library(randomForest)
Hitters.rf <- randomForest(x = X_train, y = y_train)
```

2. (1p) How many variables are used at each split. Why do you get this number?

3. (1p) Use the trained random forest to predict using the `predict` function at test set and compute the RMSE of your predictions using the `rmse` function. Add the result to your comparison table.

4. (1p) Next, use the `xgboost` R package to fit a boosted regression tree model to the training data using the `xgboost` function. Use the same predictor matrices and response vectors as above.

```
xgb <- xgboost(X_train, y_train, nrounds = 200)
```

5. (1p) Compute the test RMSE of the boosted regression tree model using the `rmse` function and add the result to your comparison table.

6. (1p) Using your comparison table, compare the test RMSEs of your single tree, random forest, and boosting model. Identify the model with the lowest test RMSE in your run and give one brief possible explanation for the differences. Answer in one or two sentences.

4 *Bagged tree and random forest implementation

This task is only necessary to get one point toward a *pass with distinction* (VG) grade. Hence, if you do not want to get a *pass with distinction* (VG) point, you can ignore this part of the assignment.

After checking your basic functions, load the optional VG checks. These use your own tree representation and do not require an exact random ensemble or test RMSE:

```
assignment_path <- paste0(
  "https://github.com/MansMeg/IntroML/blob/master/",
  "assignments/tests/assignment2/assignment2-vg.yml"
)
set_assignment(assignment_path)
```

Run the individual checks below as you implement each function. Once all four VG functions are ready, run `mark_my_assignment()` to check them together. The checks also need your basic tree functions and their helpers.

Now we can use our regression tree to do a Bagged Tree regression model. The main difference from the previous tree is that we now will draw B bootstrap samples of size N *with replacement* from our dataset, then train B trees, use them for a combined bagged prediction.

See Section 8.7 in Hastie et al. (2009), and especially Eq. (8.51) for details.

1. (2p) **Implementation (1p).** Implement a bagged tree regression model training function called `train_bagged_trees(X, y, 1, B)` using the `grow_tree()` function that return an R list of B grown trees.

   Demonstration (1p). Reuse the seeded bagging experiment below. Include the input settings, call, number of returned trees and compact actual output, with a brief explanation.

   Check: `mark_my_assignment(tasks = "train_bagged_trees")`

2. (2p) **Implementation (1p).** Implement `predict_with_bagged_trees()` using `predict_with_tree()`. Its arguments are `new_data`, a numeric predictor matrix, and `trees`, a list of grown trees. Return one prediction per observation by averaging across the trees.

   Demonstration (1p). Reuse the bagging experiment to show the inputs, call and an extract of actual predictions, with a brief explanation.

   Check: `mark_my_assignment(tasks = "predict_with_bagged_trees")`

3. (1p) Train your bagged tree regression model on the training data and predict on the test data. Try the values $B = \{1, 10, 100, 500, 1000\}$. What is the RMSE of your predictions on the test set?

   Hint! Set a seed and train 1000 trees once. For each B, average predictions from the first B trees. Test RMSE need not decrease every time B increases, so $B = 1000$ need not give the lowest RMSE in your run. The seed makes your experiment reproducible; different implementations can still produce different results with the same seed. See Figure 15.1 in Hastie et al. (2009).

Using our previous functions we can now build our own random forest regression model using our previous functions `grow_tree()` and `predict_with_tree()`. Based on the bagged tree regression model, we only need to add the parameter m.

See Section 15.2 in Hastie et al. (2009), and especially Algorithm 15.1 for details.

1. (2p) **Implementation (1p).** Create a new function `grow_tree_m()` with the additional argument m. The function should grow a tree, but at every split, it should draw a sample of size m of the covariates to make the split.

   Hint! At every node, including the root, sample m different predictors without replacement. Search for the best split using only those predictors, drawing a fresh subset for each node. With our two predictors, $m = 1$ considers one randomly selected predictor; $m = 2$ considers both, as in bagging.

   Let `X_node` and `y_node` contain the observations reaching the current node, with all predictor columns retained. The new step is:

   ```
   cols <- sample.int(ncol(X_node), size = m, replace = FALSE)
   split <- tree_split(X_node[, cols, drop = FALSE], y_node, l)

   if (!is.null(split)) {
     split$j <- cols[split$j]
   }
   ```

   If `cols = 2`, `tree_split()` reports column 1 of its input; the tree must store column 2 of the original predictor matrix. Apply the same stopping and leaf rules as in `grow_tree()` when the selected predictors admit no valid split.

   Demonstration (1p). Include a seeded example with input settings, the function call and compact actual tree output, with a brief explanation.

   Check: `mark_my_assignment(tasks = "grow_tree_m")`

2. (2p) **Implementation (1p).** Implement `random_forest(X, y, l, B, m)` to train a random forest. Draw B bootstrap samples, each containing `nrow(X)` observations sampled with replacement. Grow one tree per sample using `grow_tree_m()`, passing `l` and `m`. Return an R list of the B grown trees, in the same format as `train_bagged_trees()`. Reuse `predict_with_bagged_trees()` to average their predictions; it already calls `predict_with_tree()`.

   Complete this scaffold by adapting your bagging training loop:

   ```
   random_forest <- function(X, y, l, B, m){
     trees <- vector("list", B)
     for(b in seq_len(B)){
       # Draw a bootstrap sample of observation indices.
       # Grow a tree with grow_tree_m() and store it in trees[[b]].
       # Keep predictor subsets as matrices with drop = FALSE.
     }
     return(trees)
   }
   ```

   Demonstration (1p). Reuse the following task's seeded run to show the input settings, call and compact actual ensemble output, with a brief explanation; no extra large ensemble is required.

   Check: `mark_my_assignment(tasks = "random_forest")`

3. (1p) Call `random_forest()` on the training data with `l = 5`, $m = 1$, and $B = 100$, then use `predict_with_bagged_trees()` to predict on the test data. Set and report the seed. Include the input settings, number of returned trees, a short extract of the predictions, and the test RMSE (using `rmse()`) in your report.

   Note! With two predictors, $m = 1$ searches either `Years` or `Hits`, chosen afresh at every node, including the root; bagging searches both ($m = 2$). Either method may have lower test RMSE in a run, and differences need not be small. Differences from `randomForest` may also reflect random draws, tuning settings, and implementation choices.

References

Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The elements of statistical learning: data mining, inference, and prediction*. Springer Science & Business Media, 2009.